

Sistema di Libreria Musicale

Documento di Architettura Software e Design

Gruppo 16

Aniello Del Mauro De Girolamo

Manuel De Vivo

Simone Donato

Alessio Ferrara

1. Principi Architettureali Chiave

Il sistema di "Libreria Musicale" analizza e implementa i principali design pattern e l'architettura MVC per fornire una gestione delle playlist flessibile, solida e reattiva. La struttura di base si basa sui seguenti pilastri:

Modularità

Il sistema è nettamente suddiviso tramite l'architettura MVC. Ogni componente possiede responsabilità isolate per minimizzare gli accoppiamenti e facilitare i test di regressione.

Estensibilità

Progettato seguendo i principi SOLID. Le interfacce supportano nuove strategie di riproduzione, ordinamento avanzato e sorgenti dati senza alterare il codice esistente.

Robusta Usabilità

Integrazione di transazioni sicure, validazione immediata degli input e un gestore delle operazioni annullabili (Undo/Redo) integrato nel nucleo di controllo.

2. Gestione dei Brani (Model)

La classe Brano centralizza i dettagli di ogni singola traccia all'interno dell'applicazione, fungendo da core per l'entità principale del dominio.

Struttura e Metadati

- **Identificativo univoco (ID)** per l'indicizzazione interna sicura.
- **Metadati principali:** Titolo, Autore, Genere ed Anno.
- **Metadati dinamici:** Percorso del file fisico e durata calcolata del brano.
- **Sistema di etichettatura multipla** tramite enum tipizzato (es. *PREFERITI*, *RELAX*, *WORKOUT*).

Validazione e Caricamento

Il sistema garantisce il controllo di consistenza ed efficienza delle risorse mediante specifiche tecniche rigorose:

- **ValidazioneException:** Blocco preventivo dell'inserimento in caso di metadati incompleti, percorsi non raggiungibili o durate non conformi.
- **BranoProxy (Lazy Loading):** I dati audio pesanti vengono caricati solo al momento della riproduzione effettiva, risparmiando heap memory durante la navigazione della UI.

3. Organizzazione delle Playlist

Il sistema permette una gestione duale delle collezioni musicali per adattarsi alle diverse esigenze degli utenti:

Playlist Tradizionali

Collezioni a popolamento e gestione manuale, altamente interattive:

- Modifica diretta con operazioni rapide di aggiunta, rimozione e spostamento manuale delle tracce.
- Algoritmi di ordinamento intercambiabili basati sulle proprietà dei brani (Titolo, Autore, Genere).
- Supporto ad iteratori flessibili per definire la sequenzialità dei brani desiderata.

Smart-Playlist (Dinamiche)

Generazione automatica e reattiva basata su criteri dichiarativi:

- Popolamento dinamico basato su query e regole definite dall'utente (es: "Brani Pop pubblicati dopo il 2018").
- Aggiornamento in tempo reale: la lista si sincronizza automaticamente all'aggiunta di nuovi elementi nella libreria globale.
- Zero manutenzione manuale richiesta dall'utente finale.

4. Il Motore di Riproduzione Audio

Il sistema assicura un disaccoppiamento completo tra la sorgente audio fisica e le logiche di controllo dell'esperienza di ascolto.

L'interfaccia `Playable` astrae gli elementi riproducibili, abilitando il tracciamento in tempo reale del progresso e del volume, integrando player nativi e librerie di terze parti in maniera nativa e senza generare conflitti strutturali.

5. Design Pattern Utilizzati

Il sistema adotta diversi pattern architetturali per affrontare problemi strutturali e comportamentali comuni nell'ingegneria del software.

Pattern Creazionali & Strutturali

Singleton

Garantisce istanze uniche globali per la `Libreria` (database in memoria) e per l'`UndoManager`, che ha la responsabilità di tracciare lo storico dei comandi eseguiti.

Factory

La classe `BranoFactory` isola la complessa logica di creazione dei brani, normalizzando i metadati estratti ed istanziando correttamente i `Proxy`.

Proxy

Ottimizza l'efficienza introducendo il lazy loading dei byte audio, agendo da barriera protettiva contro consumi di RAM non necessari prima dell'ascolto.

Adapter

Il wrapper `Mp3Adapter` traduce l'interfaccia di una libreria di decoding audio esterna rendendola conforme all'interfaccia interna `Playable`.

Pattern Comportamentali Core

Observer

Notifica istantaneamente le View (UI) in caso di variazione dello stato della riproduzione (traccia cambiata, stop, pause) o in caso di modifiche alle playlist.

Strategy

Definisce le strategie di playback intercambiabili a runtime: Sequenziale, Shuffle (riproduzione casuale) e Loop (ripetizione della singola traccia).

Command

Inquadra le azioni distruttive o critiche come comandi reversibili per abilitare un solido meccanismo di Undo/Redo all'interno dell'interfaccia.

State

Rappresenta gli stati del player audio (PlayingState, PausedState, StoppedState), incapsulando il comportamento da applicare ad ogni cambio di stato.

6. Flusso Architettuale MVC

La seguente tabella illustra i livelli del pattern Model-View-Controller adottati all'interno della struttura dell'applicazione:

Componente	Classi Coinvolte	Responsabilità Principale	Interazione nel Flusso
Model	Brano, Playlist, Libreria	Gestisce i dati di business, le regole di validazione e lo stato del database musicale.	Modifica lo stato e notifica la View tramite eventi (pattern Observer).

Componente	Classi Coinvolte	Responsabilità Principale	Interazione nel Flusso
View	LibreriaView, PlayerView	Renderizza l'interfaccia utente grafica (UI) e rileva gli input dell'utente in ascolto.	Invia gli input al Controller e si aggiorna reattivamente.
Controller	Libreria Controller	Insegue il flusso applicativo e mappa le azioni utente in comandi specifici sul Model.	Intercetta le gesture inviate dalla View e invoca azioni sul Model o meccanismi di Undo.

7. Metodologia di Sviluppo (Agile)

Il ciclo di sviluppo ha seguito in maniera rigorosa il framework Agile (Scrum/Kanban), passando per le seguenti fasi:

- **Dalle Epiche al Backlog:** Traduzione delle grandi User Epics in User Stories azionabili e indipendenti all'interno del Product Backlog.
- **Definizione Requisiti:** Utilizzo sistematico della struttura: *"Come [utente], voglio [funzionalità], affinché [beneficio]"*.
- **Acceptance Criteria & DoD:** Inclusione di precisi criteri nel formato formale *Given/When/Then* per stabilire chiaramente la Definition of Done (DoD) per ciascun task affrontato.
- **Sprint Planning:** La velocity prevista per il First Sprint Backlog è stata di 15 Story Point, permettendo la consegna iterativa di incrementi architetturali.